

Electrolux Knowledge Assessment - Answers

Question 3

A simple way to control the power of a resistive load is to use a soft-start.

For a purely resistive DC load, the circuit can be controlled using PWM. The microcontroller starts with a low duty cycle and gradually increases it until the desired power is reached. As for a resistive AC load, the power can be increased gradually by controlling how much of each AC cycle is applied to the load.

If the load is inductive, the same general control method can still be used, but additional protection is required. An inductive DC load usually needs a flyback diode because interrupting the current can generate a high-voltage spike. For an AC inductive load, the current can lag behind the voltage, so the switching timing must take this phase difference into account, and additional protection against voltage transients may also be necessary.

Question 4

Circuit Comparison

Circuit 1

R_{19} is a pull-down resistor. C_{14} and C_{15} provide hardware filtering against high-frequency noise and short transients. R_{20} limits the current that can flow toward the protection circuit, while the BAT54S *Schottky* diode clamps the signal to approximately the supply rails.

Circuit 2

R_{56} , R_{57} and R_{58} form a total series resistance of $450\text{ k}\Omega$, and form a voltage divider with R_{59} . For DC or low-frequency analysis, the attenuation is approximately $V_s = V_{in}/81$. C_2 is connected in parallel with R_{59} , working as a low-pass filter. Therefore, the exact attenuation of an AC signal depends on frequency. R_{61} provides additional current limiting, while the BAT54S *Schottky* diode protects the microcontroller input from voltages outside the intended range.

Microcontroller Pin Configuration and Software Filtering

The outputs of both circuits can be connected to digital GPIO inputs of the microcontroller. The internal pull-up and pull-down resistors should be disabled.

Circuit 1

The output can normally be read directly as a digital HIGH or LOW level. The required software filtering depends on the source connected to the circuit. If the input signal comes from a mechanical switch, a debounce filter should be used. If the signal comes from a clean electronic source, additional filtering may not be necessary. If short glitches are expected, the software can require the new level to be present for several consecutive samples before accepting the transition.

Circuit 2

If the input is DC, it can be interpreted in the same way as a normal digital input. If the input is AC, the software should detect repeated transitions, using either periodic polling or

GPIO edge detection. The signal can be considered present when valid transitions continue to occur and considered absent when no transition is detected for a defined timeout.

Design Document - Switch Status Reading

Note: Since the original document does not indicate which circuit is circled in green, one of the presented circuits was chosen arbitrarily.

The purpose of this design is to define the hardware interface and software behavior required to reliably determine the state of the switch connected to Circuit 1. The implementation should provide a stable switch status to the application while rejecting contact bounce and short-duration disturbances.

Assuming that closing the switch applies a valid logic-high voltage to the input, the expected states are

Switch OPEN → GPIO LOW
Switch CLOSED → GPIO HIGH

The microcontroller pin can be configured as a digital input. The switch status may be acquired by periodic polling. An interrupt is not required for normal operation because the switching event is slow compared with the microcontroller execution rate.

The switch input can be sampled periodically. It is advisable to separate the raw GPIO value from the validated switch state so that temporary transitions do not immediately affect the rest of the application. If the input is connected to a mechanical switch, a debounce filter is needed.

The following block shows an example for this implementation

```
#define SWITCH_DEBOUNCE_TIME_MS 20

static bool previous_raw_state = false;
static bool filtered_switch_state = false;
static uint16_t debounce_counter = 0;

void Switch_Task(void){
    bool raw_state = GPIO_Read(SWITCH_PIN);

    if (raw_state == previous_raw_state){
        if (debounce_counter < SWITCH_DEBOUNCE_TIME_MS){
            debounce_counter++;
        }
        if (debounce_counter >= SWITCH_DEBOUNCE_TIME_MS){
            filtered_switch_state = raw_state;
        }
    }
    else{
        previous_raw_state = raw_state;
        debounce_counter = 0;
    }
}
```

The rest of the application can use the filtered state instead of reading the GPIO directly.

Question 6

I would try to store all the registers in some static memory during the fault handler, if possible. Otherwise, if I had a debugger, I could try simulating the high-interrupt-load scenario while running the debugger and break on the fault handler to inspect the registers.

Inspecting the program counter would probably be the most useful, since that is generally the case with many bugs. After that, I would also look at the fault registers, since the HardFault

could have escalated from one of the more specific faults. I am also assuming that all the specific faults are handled, so this HardFault is not simply a more specific fault that was not handled.

Even though no stack overflow was identified, it could be that the linker script was configured incorrectly and the stack memory was mapped incorrectly for the hardware, resulting in a bus fault or memory protection fault, so I would double-check that.

It could also be some kind of race condition. Given that there are a lot of interrupts occurring and UART runs asynchronously, there may be things happening concurrently, even if the processor receiving the messages does not have any actual parallelism.